

TP1 - R2.08

Les fonctions:

L'utilisation des fonction permet de structurer ses programmes, de réutiliser les parties communes et de mieux les maintenir. Certaines sont prédéfinies et déjà vues pendant les TPs R107. Nous allons dans ce TP à écrire, correctement, nos propres fonctions

- Dans un script `program\_square.py` :
 - Ecrire une fonction square qui prend en paramètre une variable `x` et retourne x^2
 - Ajouter la documentation de cette fonction
 - Ajouter plusieurs tests avec un affichage du résultats pour validation

```
home > ruhollah_sherzad > program_square.py > ...
1 def square(x):
2     """Retourne le carré de x."""
3     return x ** 2
4
5 # Partie test
6 if __name__ == "__main__":
7     test_values = [16, 24, 0, 4.5]
8     for val in test_values:
9         print(f"square({val}) = {square(val)}")
10
```

```
square(16) = 256
square(24) = 576
square(0) = 0
square(4.5) = 20.25
ruhollah_sherzad@j001-15:~/Documents/SAE15$
```

- Dans un script `program_pi.py`
 - Ecrire une fonction `mypi()` avec un argument `precision`. Cette fonction retourne une valeur approchée de π , sachant que : $\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$ (remplacer donc ∞ avec `precision`)

Attention, cette approximation est une quantité *réelle* : utilisez des *flottants*. Pour cela vous avez besoin de la fonction « racine carrée » dont le nom en Python est `sqrt()` et qui est accessible depuis le module `math`.

- Par ailleurs, écrire un programme principal qui saisit un entier `n` positif non nul, qui appelle la fonction `mypi()` et qui affiche vos résultats ainsi que l'erreur relative (en pourcentage) entre votre valeur de π et celle fournie par le module `math`

```

import math

def mypi(precision):
    """Retourne une approximation de  $\pi$  en sommant 'precision' termes de la série de Bâle."""
    somme = sum(1 / (i ** 2) for i in range(1, precision + 1))
    return math.sqrt(6 * somme)

if __name__ == "__main__":
    n = int(input("Entrez un entier positif non nul : "))
    approx_pi = mypi(n)
    erreur_relative = abs((math.pi - approx_pi) / math.pi) * 100

    print(f"Approximation de  $\pi$  avec {n} termes : {approx_pi}")
    print(f"Valeur réelle de  $\pi$  : {math.pi}")
    print(f"Erreur relative : {erreur_relative:.6f}%")

```

```

ruhollah_sherzad@j001-15:~/R208/tp1/program_pi.py$ /usr/bin/env /bin/python3 /home/ruhollah_sherzad/.vscode/extensions/ms-python.debugpy-2024.0.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 35515 -- /home/ruhollah_sherzad/R208/tp1/program_pi.py/program_pi.py
Entrez un entier positif non nul : 24
Approximation de  $\pi$  avec 24 termes : 3.1023765763598083
Valeur réelle de  $\pi$  : 3.141592653589793
Erreur relative : 1.248287%
ruhollah_sherzad@j001-15:~/R208/tp1/program_pi.py$ cd /home/ruhollah_sherzad/R208/tp1/program_pi.py ; /usr/bin/env /bin/python3 /home/ruhollah_sherzad/.vscode/extensions/ms-python.debugpy-2024.0.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 54259 -- /home/ruhollah_sherzad/R208/tp1/program_pi.py/program_pi.py
Entrez un entier positif non nul : 69
Approximation de  $\pi$  avec 69 termes : 3.1278227179851794

```

Les modules

La portée des variables, les espaces de noms

Essayez dans l'interpréteur Python, et commentez :

>>> import math ⇒ import module de math dans notre terminal (python)
 >>> dir(math)⇒ affichage de tous les attributs math dans module
 >>> help(math) ⇒ affichage de documentation de module qui nous aide à bien comprendre

>>> math.pi ⇒ cette ligne nous affiche pi ou 3.14
 >>> math.pi.as_integer_ratio() ⇒ la commande nous retourne une fraction qui représente π sous forme de ratio d'entiers.
 >>> import datetime⇒ import la module datetime, aide à manipuler l'heure et date
 >>> datetime.date.today()⇒ Cette commande retourne la date actuelle.

- Testez le petit script scopes.py suivant et commentez :

Pour comment cette script on peut dire

-on definir une variable x

- def f1() nous affiche le message appelle puis la valeur

x

Ce script illustre la différence entre les variables globales et locales en Python. La fonction `f1()` utilise `x` global, tandis que `f2()` crée une `x` locale sans affecter `x` global. La fonction `f3()` modifie `x` globalement et crée `y` localement, entraînant une erreur lorsque `y` est utilisé en dehors de `f3()`.

```
x = "valeur globale"

def f1() :
    print ("--Appel f1()")
    print (" x dans f1 :", x)

def f2() :
    print ("--Appel f2()")
    x = "valeur locale"
    print (" x dans f2 :", x)

def f3() :
    print ("--Appel f3()")
    global x
    x = "valeur modifi e"
    y = 12
    print (" x dans f3 :", x)
    print (" y dans f3 :", y)

print ("x au niveau global :", x)
f1()
print ("x au niveau global :", x)
f2()
print ("x au niveau global :", x)
f3()
print ("x au niveau global :", x)
print ("y au niveau global :", y)
```

Les modules

Jusqu'à maintenant, tous les scripts que nous avons écrits n'étaient constitués que d'un seul fichier. Or quasiment tous les langages proposent et préconisent le développement multi-fichiers. Python propose une technique particulièrement simple : le module

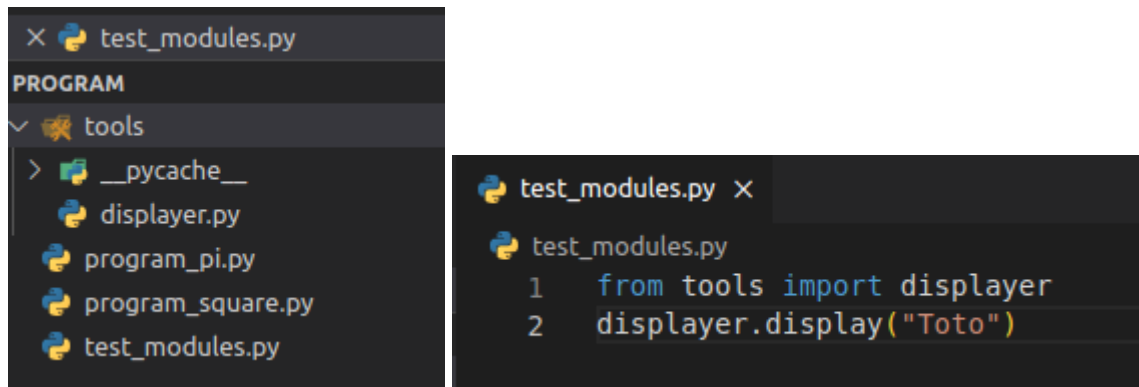
Créons par exemple un module `displayer`, contenant une fonction `display()` ainsi que son code d'auto-test. C'est tout simplement un fichier source Python `displayer.py` contenant :

```
program_pi.py  displayer.py x
displayer.py > ...
1  #!/usr/bin/python3
2  # -*- coding: utf-8 -*-
3
4  """Display Module"""
5
6  def display(x):
7      """Affiche la valeur passée en paramètre."""
8      print(x)
9
10 if __name__ == "__main__":
11     display(42)
12     display("Toto")
13
```

```
File "/home/ruhollah_sherzad/.vscode/extensions/ms-python.debugpy-2024.0.0-linux-x64/debugpy/_vendored/pydevd/_pydevd_bundle/pydevd_runpy.py", line 124, in _run_code
    exec(code, run_globals)
File "/home/ruhollah_sherzad/R208/tp1/program_pi.py/scopes.py", line 27, in <module>
    NameError: name 'y' is not defined
ruhollah_sherzad@j001-15:~/R208/tp1/program_pi.py$ cd /home/ruhollah_sherzad/R208/tp1/program_pi.py; /usr/bin/env /bin/python3 /home/ruhollah_sherzad/.vscode/extensions/ms-python.debugpy-x64/bundled/libs/debugpy/adapters/.../debugpy/launcher 42019 -- /home/ruhollah_sherzad/program_pi.py/displayer.py
42
Toto
ruhollah_sherzad@j001-15:~/R208/tp1/program_pi.py$
```

Maintenant, si dans un script, on a besoin de la fonction `display()` du module `displayer`, il suffit de l'importer au préalable.

- Ecrire un script `test_modules.py` qui importe ce module et le test. Rappelez vous qu'il y a deux façons d'importer ...
- Afin de créer un package, déplacer le module vers un répertoire `tools`, Quels changement faut réaliser pour que votre script `test_modules.py` continue à fonctionner ?



Documentation:

- On l'a déjà dit, documenter ses scripts est important. Mais Python offre beaucoup plus. Par exemple :
 - [Commentez la partie `\_\_main\_\_`](#)
On peut dire que cette condition vérifie si le script est exécuté **directement** (et non importé comme un module).
Si c'est le cas, le bloc de code suivant s'exécute.

```
print("\n+++ Documentation stockée dans un attribut de la fonction:")
print(ras.__doc__)
```

Affiche la **documentation** de la fonction `ras`, stockée dans `ras.__doc__`.
Cela permet de tester et afficher le docstring de la fonction

```
print("\n+++ Exemple d'appel à la fonction:")
```

- La documentation issue du code source est ainsi directement accessible via l'aide en ligne :

```
>>> from module import ras
>>> help(ras)
Help on function ras in module my_module:

ras()
    Cette fonction est bien documentée mais ne fait presque rien.
```

Lint et formatter

- A l'aide de la documentation <https://code.visualstudio.com/docs/python/linting> et https://code.visualstudio.com/docs/python/editing#_formatting :
 - Quelle est la différence entre un linter et un formatter ?
Lint⇒ Analyse le code et signale les erreurs potentielles, les mauvaises pratiques et les incohérences de style.
Formatter⇒ Reformate automatiquement le code pour respecter un style défini ex indentation, espaces ;
Uniformiser l'apparence du code sans modifier sa logique
 - Installer et configurer le linter pylint et le formatter black.

```
ruhollah_sherzad@j001-15:~$ pip install pylint black
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pylint in ~/.local/lib/python3.10/site-packages (3.3.5)
Requirement already satisfied: black in ~/.local/lib/python3.10/site-packages (25.1.0)
Requirement already satisfied: tomlkit>=0.10.1 in ~/.local/lib/python3.10/site-packages (from pylint) (0.13.2)
```

- Tester les deux